C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural Networks

David Gonzalez[1], Joshua T. Guerin, Ph.D.[2]
Mathematical Sciences
University of Northern Colorado
Greeley, CO 80639
[1]********@bears.unco.edu
[2]********@unco.edu

Abstract

As Artificial Intelligence and generative AI continue to engage the public in ways that mirror other critical advances (e.g., the PC and smartphone revolutions as well as the broader Internet) public interest has turned more and more towards applications of the underlying technologies and architectures that power them. While machine learning technologies like LLMs have a profound capacity to integrate AI into modern life, the underlying artificial neural network technologies present significant concern for both short and long-term sustainability: data center footprint, waste heat and noise, and infrastructure/resources for energy and clean water.

At the same time, advanced declarative languages have made remarkable headway towards AI solutions are considerably more efficient. In particular, Answer Set Programming languages are designed to solve computationally difficult (e.g., NP-Complete and NP-Hard) problems with limited available resources. In this paper we outline how these languages can be used to model artificial neural network architectures, like those used in LLMs, in the modeling language Clingo. We also introduce the C-MCPN (Clingo McCulloch-Pitts Network) Framework and Editor– a tool largely written in Python and Clingo. C-MCPN was developed to support our work in generating more efficient neural networks by providing a graphical interface to work with these models, using Clingo as a back-end for both representation and inference.

1 Introduction

Artificial Intelligence systems have had a significant impact on the modern world over the decades. Some of the many advancements across various fields include conversational agents, marketing, medicine, and insurance [15]. Unlike the impacts of AI in previous decades, general public awareness has skyrocked due to advances in generative AI (GEN-AI), in particular techniques from machine learning (ML) like the artificial neural networks (ANNs) that power contemporary large language models (LLMs).

While these AI systems have the potential for significant advancement, the underlying technologies come with significant drawbacks: footprints of data centers (which has garnered particular public interest–[14]), worsening use of finite resources such as fresh water and electricity, and the potential for significant environmental and health impacts [9].

Conversely, Answer Set Programming (ASP) languages often live at a different end of the research spectrum in terms of their efficiency. These languages are designed to solve NP-Complete and NP-Hard problems in a way that is fast and efficient on limited hardware such as consumer computers. Our current work involves exploring the use of ASP languages to accelerate the processes of learning and inference, with a hope of cutting into the resources needed for these technologies to continue growing in use.

In this paper we demonstrate that ANNs, specifically McCulloch-Pitts Networks (MPNs) can be used to model and conduct inference over McCulloch-Pitts neuroons [12]. We also introduce `C-MCPN`: The *Clingo McCulloch-Pitts Network Editor*. C-MCPN is a tool that we have developed to assist in the exploration of ASP-based networks, and allows the user to model network architectures, including the underlying representations and inference over ANNs.

2 Background

Our work is related to the merging of ANN technology with more efficient Answer Set Programming languges. In this section we will provide a brief background on both subjects, motivating our work on C-MCPN.

2.1 Artificial Neural Networks (ANNs)

Artificial Neural Networks (ANNs) are a class of machine learning models inspired by the structure and function of biological neural networks. The basic building blocks of ANNs are artificial neurons, and their origins can be traced back to the work of McCulloch and Pitts in the 1940s [12]. These early models were designed to mimic the behavior of biological neurons, using a simple thresholding function to determine whether a neuron would fire based on its

inputs. The name for the C-MCPN was inspired by this early work, though our implementation is not a pure McCulloch-Pitts model which uses a threshold function as a simple activation function. A more generalized version of a threshold logic unit (TLU) which can be seen in Figure 1.

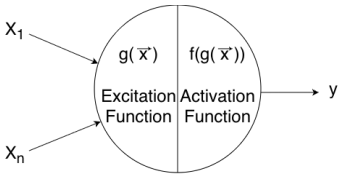

Figure 1: A diagram of a 2-input Threshold Logic Unit (TLU)

The idea of an ANN evolved significantly over the decades, with more recent developments relating closely to the research of graph neural networks (GNNs) starting from the early 2000s [8, 21]. Since then, there has been lots of research into different ways to build a model from creating knowledge graphs to dynamic graph structures to Markov-Chain models such as PageRank-based ones [21]. One of the most popular and widely used types of ANNs are feedforward neural network, which combined with the idea of attention mechanisms have led to the development of transformer-based models such as BERT and GPT [19, 5, 2]. Its no secret that these models have had a significant impact on the rest of the world, but studies have shown that they have very significant drawbacks. These come in the form of resource use as discussed briefly in Section 2.1.1, lack of interpretability and explainability [16], and issues with bias and fairness along with a possibility for misuse [2].

Despite these drawbacks, ANNs have been widely adopted in various applications. One of the more common forms of ANN is the feedforward neural network, which consists of layers of interconenected neurons that process data in a single forward direction [7]. An example of a simple feedforward neural network can be seen in Figure 2. The input layer holds the data which is passed forwards to two hidden layers, and finally to the output layer which produces the final result. Notably, these models do not necessarily learn; learning primarily comes from a training process where weights are adjusted based on the error between predicated and actual outputs, typically done using backpropagation and gradient descent which can be computationally expensive. These more adaptable models are more powerful than the original non-learning McCulloch-Pitts model, however both are based on the same underlying structure of a feedforward network of neurons.

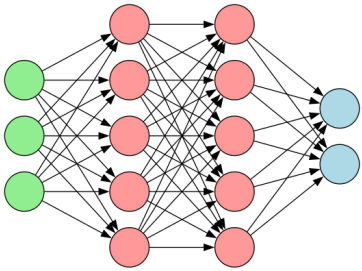

Figure 2: A diagram of a simple feedforward neural network

2.1.1 Environmental Impacts

The introduction of big data (in particular by scraping the broader Internet) changed the landscape for AI, allowing statistical models to flex their strength, in particular because these models require massive amounts of data for training purposes [10, 1]. This incredible need for information highlights an important drawback in the form of *data efficiency* [4]. This notable lack of efficiency translates to a significant need in terms of resources.

As an example, let us consider a the cost to train a single model, ChatGPT-3. According to Vries [20], the training of GPT-3 required around 1.3 million kWh *just* to be trained. With some quick, back-of-the-envelope calculations, we can put this into perspective. Consider that in 2022 the average American household used approximately 10,791 kWh [18]. Dividing these out we find that the model's energy use was proportional to:

$$1{,}300{,}000 \text{ kWh} \times \frac{1 \text{ household}}{10{,}791 \text{ kWh/year}} \approx 120.5 \text{ households.}$$

Similarly, we could produce a rough, *lower bound* of how much water was likely used for training. A termoelectric power station can use approximately 43.8 L/kWh of water [11]. That would put the *indirect* water costs of training GPT-3 at approximately 56,940,000 L, approximated by:

$$1{,}300{,}000 \text{ kWh} \times 43.8 \frac{\text{L}}{\text{kWh}} = 56{,}940{,}000 \text{ L}$$

Given that an adult male averages approximately 4L of fresh water, that number represents sufficient water to sustain a lower bound of approximately 39,000 adults for a year [13]:

$$56{,}940{,}000 \text{ L} \times \frac{1 \text{ adult}}{4 \text{ L/day} \times 365 \text{ days}} \approx 39{,}000 \text{ adults}$$

To put this number into perspective, this would likely be sufficient to provide fresh water to approximately the city of Brighton, CO, for a year.

Note that this figure includes only the indirect costs to generate the electricity– massive cooling costs for data center equipment would likely add significantly to our calculation. While these calculations are remarkably overly-simplified, we hope that it puts into perspective how the technology is already unsustainable. This highlights the dire need for more efficient AI and ML technologies [20]. The discovery of models that are sufficiently sustainable to curb this resource use is an open problem.

We believe that one path towards a solution is to combine different AI methodologies, in particular those technologies that are known for higher efficiency. One possible candidate would be the use of satisfiability solvers, in particular modern Answer Set Programming languages, to accelerate the learning and inference processes.

2.2 Answer Set Programming (ASP)

Answer Set Programming serves as an amalgamation of Prolog-style (i.e., quantified logic) with powerful advances in satisfiability (SAT) solvers that occurred throught the 1990s and into the 2000s, expanding significantly in terms of the model sizes they are capable of processing and computational efficiency. In this section we will discuss the basic syntax and semantics of the ASP language, Clingo, which serves as a technological foundation of C-MPCN [6, 3].

2.2.1 Syntax and Semantics

The atomic units in an ASP dialect such as Clingo are *literals*, which describe ideas and *facts* that serve as statements that are held to be true. The basic syntax is the application of a *property* to a literal, `property(atom)`. E.g.,

```
red(apple).
```

is a complete statement (and program), attributing a property of red to an idea named `apple`. Such statements are *purely declarative*, providing a *description* of the problem (vs. specific processor instructions). I.e., we describe *what* needs solving vs. *how* to solve the problem in traditional imperative/algorithmic contexts.

Further knowledge is*inferred* using aggregate statements known as *rules*, taking the form: `consequent :- antecedent`. This mirrors an *implication* from formal logic, as the `:-` symbol is read equivalently to $\leftarrow$. In such a statement the consequent *must* be true whenever the antecedent is true. E.g., consider the statement:

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

This describes a preference over what fruit will be eaten. I.e., "If a fruit is red and ripe, then we eat the fruit."

In this case rather than the literal `apple`, we use a *variable* to allow for inference. I.e., The `Fruit` can be *any* contextually-relevant literal in the program, so long as it possesses the required properties. Note that this functions differently from its analogous notion in imperative language–rather than a pattern to be matched variables typically serve as typed memory locations that are manipulated by the program.

2.3 ANNs in Clingo

Creating a representation of an ANN in Clingo is a non-trivial task, as the language is not designed for this purpose. To do so, we first needed a framework for representing the structure of an ANN in Clingo, which we call the C-MCPN Framework. This framework allows us to represent the structure of an ANN using a combination of facts and rules, and to specify the activation function of each neuron using rules that define how the values of the neurons are calculated based on their inputs. The representation in Figure 1 is the one used for the neurons in the C-MCPN Framework, where the excitation function $g(\vec{x})$ is defined by the type of *gate*, and the activation function is handled by edges carrying values from input to output. As such, the illustrative domain we have selected is digital logic for hardware development. This domain has several attractive features including simplicity and that it is commonly taught in most CS programs. This also allows us to develop networks of arbitrary complexity for experimentation.

For our example network we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the McCulloch-Pitts definitions of these gates, and their truth tables. Using a combination of these and more gates, one is able to construct complex logical operations[1]. Moreover, we believe that the C-MCPN Framework has the potential to be used for a wide range of applications in the future, especially if its capabilities are further explored and developed.

Did this table come from somewhere that needs citing, or did you come up with the layout and content? :- I came up with it, I spent a ton of time stitching together a couple of other tables I had made. I came up with all of the content, but it didn't seem too complex so I didn't think it needed a citation or anything.

[1] An elegant and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

Table 1: McCulloch-Pitts Neuron Definitions and Truth Tables.

AND			**OR**			**XOR**		
$g(\vec{x}) = x_1 \cdot x_2,\ \theta = 1$			$g(\vec{x}) = x_1 + x_2,\ \theta = 1$			$g(\vec{x}) = \lvert x_1 - x_2 \rvert,\ \theta = 1$		
Activation: $f(\vec{x}) = \begin{cases} 0 & \text{if } g(\vec{x}) < \theta, \\ 1 & \text{if } g(\vec{x}) \geq \theta. \end{cases}$								
AND			OR			XOR		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

The representation of such an artificial neuron requries both the *structure* designation, as well as the function used to *activate* the neuron. When activated the neuron will propagate a singnal to its output given the provided input signals.

We will use an `AND` gate to illustrate how we represent an artificial neuron:

```
% type("TYPE")
type("AND").

% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :- values(In1), values(In2),
                              Out = (In1 * In2).
```

In this case the *structure* of the neuron is indicated by the parameters: in the case of an `AND` gate this would require two input parameters and a single output. The *activation function* is encoded directly in the rule structure using the arithmetic equivalent provided in Table 1.

By specifying the *connectivity* of these neuron definitions, we can forge the connections of a network. In this case we will create a simple 1-bit adder circuit:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").
```

```
% There can be at most 2 inputs.
{ edge(src, "XOR_1") : neuron("XOR", "XOR_1") } 2.

% val(neuron_id, value).
val("INPUT_1", 1).
```

Note the *cardinality constraint*, enforcing at most two inputs. These constraints allow additional numeric constraints to be embedded in rules. Syntactically, this can be summarized as `l { } u`, where `l` and `u` prescribe (optional) upper and lower bounds on the contents.

The value inference program performs two important roles. It activates each gate by allowing the edges between neurons to carry values, and it assigns each individual neuron a value based on its gate type. The following lines are from the value inference program, and show how this is implemented:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).

% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.
```

Note the first fact which sets the domain of values to be binary and the constraint that ensures each neuron has exactly one value. Most importantly, the rule which carrys the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values of the neurons should be. This is a key insight we had during development.

3 C-MCPN

In order to better explore the domains described in this paper we developed the C-MCPN software package to automate our workflow. The system itself is designed around a standard (unix-like) terminal use and is extended into a graphical user interface (GUI).

The core functionalities of C-MCPN are built on the ASP language Clingo, as described in the preceding sections. The editor and interface were constructed in Python using the DearPyGui library, allowing us to take advantage of its node-based editing features.

Building the editor in this way also allowed us to easily integrate it with the C-MCPN Framework, as we could directly use the same two-layered structure for elaborating both the value of each neuron as well as the network architecture. This was an important design choice, as it allows the editor to be easily extended in the future to support a wider range of applications.

In terms of the reorganization algorithm we used a modified version of the Sugiyama algorithm, taking just the layer assignment step and a simplified node positioning step [17].

3.1 C-MCPN Framework

From what we could find in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework, which can be seen in Figure 3, uses three main program types:

- **Gate Definitions Program:** Files are in the `C-MCPN/gates/` directory.
- **Network Structure Program:** Files are in the `C-MCPN/networks/` directory.
- **Value Inference Program:** Files are in the `C-MCPN/base/` directory.

The files in the gate definitions program define the behavior of each gate, including the number of inputs it should have. Every default gate we included also has a special header used to send meta-information to the C-MCPN Editor. (E.g., The `AND` formulation specified in Section 2.3.

3.2 C-MCPN Editor

In the `app/` directory of the project, we have the code for the C-MCPN Editor. The main features of the editor include the ability to add/remove neurons, connect/disconnect them, copy/paste them, and run inference on the network.

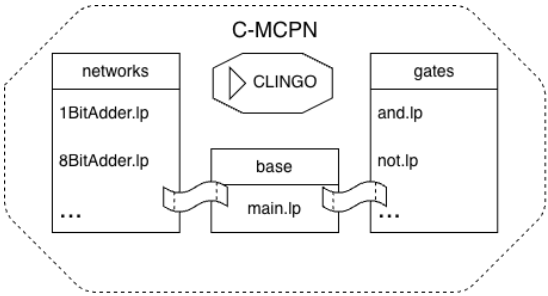

Figure 3: A visualization of the programs involved in the C-MCPN Framework.

The editor allows the files to be edited directly from the interface, transforming it into more of an interactive development environment (IDE) for logical networks. It includes a preview feature which allows users to see the structure of the network in ASP format before and after running the inference with simple syntax highlighting; helpful for debugging and understanding how the network is functioning. There is also an option to create new logic gates that the neurons can use, allowing for a wider range of applications.

Custom networks are saveable to JSON or ASP format and loadable from JSON format, making it easy to work on them over multiple sessions and share them with others. Saving the network to ASP format allows direct use with the C-MCPN Framework from the terminal instead of IDE.

The inputs for the inference use a Python script, allowing users to create complex inputs for their custom networks. The output can also be parsed with savable Python scripts built directly in the editor, allowing the user to choose how they want to process the results. Finally, it is worth noting that the editor includes a reorganization algorithm which automatically rearranges the neurons to make the network easier to read and understand after changes are made. This algorithm is based on the Sugiyama algorithm for layered graph drawing [17].

A screenshot of the editor can be seen in Figure 4, showing a sample 1-bit adder network.

3.3 AI-Assisted Development

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were conscious decisions by the authors. Each line of code generated by Claude was reviewed, edited,

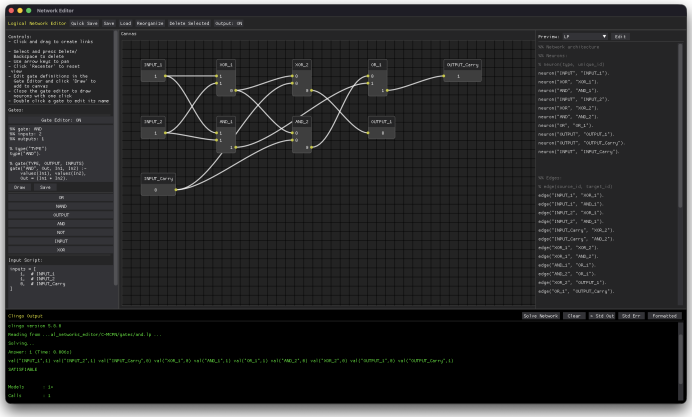

Figure 4: A screenshot of the C-MCPN Editor, showing a sample 1-bit adder network.

and approved by the authors before being added to the codebase. In order to maintain a supervisory-level of control the agent was not given direct access to the codebase or the repository. The file that Claude had input in are contained in the `app/` directory of the project, allowing users the ability to inspect these files if they so desire.

Use of Claude was employed only in the above described sections in order to accelerate the development process under circumstances of limited resources.[2]

4 Evaluation

Once the screenshot is removed this section may be something we can kill for space. The 1-bit adder example network was used to evaluate the framework for correctness. Its truth table was compared to the expected output for all permutations of possible valid inputs. The verified network can be seen in in Figure 5.

The C-MCPN Framework works well for logical operations with a fixed number of inputs and outputs, and the C-MCPN Editor makes it easy to construct and visualize these networks. There were two key insights gained throughout the creation of this framework which are the importance of the edges carrying values and the minimalistic fact structure which improves the

[2]The authorship of this paper is without the assistance of any AI system, including planning, writing, editing, and repository maintenance.

performance of the network. This framework is a novel approach to neural network representation within a logical programming paradigm like ASP.

The C-MCPN Editor also shows potential as a powerful educational tool for understanding both neural networks and logic programming. Showing a visualization of the network structure along with a live preview of the code necessary to create it is a great way to help users relate the two ideas.

Editor screenshots should be moved to the preceding section.

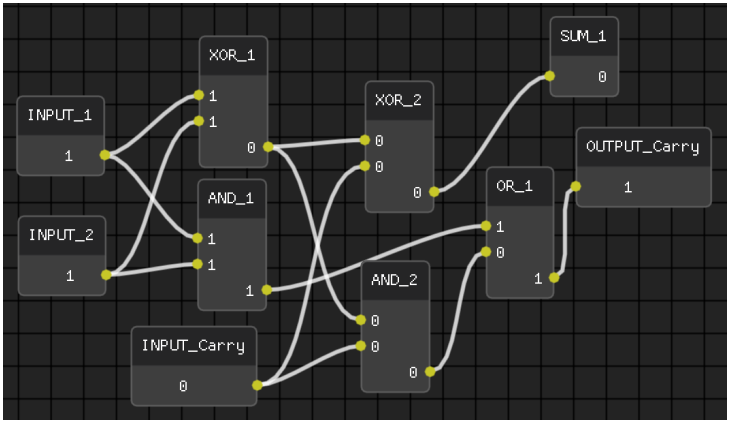

Figure 5: A screenshot of a 1-bit adder network in the C-MCPN Editor.

4.1 Limitations

This work is a proof of concept for the C-MCPN Framework and Editor, and as such there are some limitations to be aware of. The framework has no cycle detection nor has it been tested on recurrent networks. This is likely to cause issues with the value inference program, as it relies on the values being propagated along the edges. The framework has also not been tested on large networks, and it is unknown how scale will affect its performance. It is also worth noting that Clingo does not have native support for floating point numbers, limiting the potential applications of easy to understand networks.

5 Conclusion

The C-MCPN Framework and Editor provide a novel approach to representing and working with logical networks within the paradigm of logic programming. The framework allows for the construction of logical networks using simple facts and rules, while the editor provides an interactive environment for building

and visualizing these networks. Full instructions for using the editor can be found in the README of the project, and we encourage readers to try it out for themselves at `https://github.com/David-Gonzalez-Sua/logical_networks`. Importantly, the editor is designed with accessibility in mind, and we hope it can be used by a wide range of people to learn about these concepts.

5.1 Future Work

There is a lot of potential for future work in this area, both in terms of improving the C-MCPN Framework and Editor, and in exploring new applications for logical networks. For example, we have begun work on a new framework for an adaptive neural network which can learn on data that uses the C-MCPN Framework as its skeleton. This framework is in its early stages, but we hope to also add it as a feature in the editor in the future.

References

[1] Bikram Pratim Bhuyan et al. "Neuro-symbolic artificial intelligence: a survey". In: *Neural Computing and Applications* 36.21 (2024), pp. 12809–12844. DOI: `http://dx.doi.org/10.1007/s00521-024-09960-z`.

[2] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. DOI: `http://dx.doi.org/10.48550/arXiv.2005.14165`.

[3] Pedro Cabalar et al. "On the Semantics of Hybrid ASP Systems Based on Clingo". In: *Algorithms* 16.4 (2023), p. 185. DOI: `http://dx.doi.org/10.3390/a16040185`.

[4] Andrew Cropper, Sebastijan Dumančić, and Stephen H. Muggleton. "Turning 30: New Ideas in Inductive Logic Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. 2020, pp. 4833–4839. DOI: `http://dx.doi.org/10.24963/ijcai.2020/673`.

[5] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: `http://dx.doi.org/10.18653/v1/N19-1423`.

[6] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82. DOI: `http://dx.doi.org/10.1017/S1471068418000054`.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[8] M. Gori, G. Monfardini, and F. Scarselli. "A new model for learning in graph domains". In: *Proceedings of the 2005 IEEE International Joint Conference on Neural Networks, 2005.* 2 (2005), pp. 729–734. DOI: http://dx.doi.org/10.1109/IJCNN.2005.1555942.

[9] Yuelin Han et al. *The Unpaid Toll: Quantifying and Addressing the Public Health Impact of Data Centers*. 2025. DOI: http://dx.doi.org/10.48550/arXiv.2412.06288.

[10] J. Mathew Helm et al. "Machine Learning and Artificial Intelligence: Definitions, Applications, and Future Directions". In: *Current Reviews in Musculoskeletal Medicine* 13.1 (2020), pp. 69–76. DOI: http://dx.doi.org/10.1007/s12178-020-09600-8.

[11] Pengfei Li et al. *Making AI Less "Thirsty": Uncovering and Addressing the Secret Water Footprint of AI Models*. arXiv:2304.03271. 2023. DOI: http://dx.doi.org/10.48550/arXiv.2304.03271.

[12] Warren S. McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133. DOI: http://dx.doi.org/10.1007/BF02478259.

[13] Institute of Medicine. *Dietary Reference Intakes for Water, Potassium, Sodium, Chloride, and Sulfate*. Washington, DC: The National Academies Press, 2005. DOI: http://dx.doi.org/10.17226/10925.

[14] Engineering National Academies of Sciences and Medicine. *Implications of Artificial Intelligence–Related Data Center Electricity Use and Emissions: Proceedings of a Workshop*. Ed. by Anne Frances Johnson and Kasia Kornecki. Washington, DC: The National Academies Press, 2025. DOI: http://dx.doi.org/10.17226/29101.

[15] Albert Nössig, Tobias Hell, and Georg Moser. "Rule learning by modularity". In: *Machine Learning* 113.10 (2024), pp. 7479–7508. DOI: http://dx.doi.org/10.1007/s10994-024-06556-5.

[16] Cynthia Rudin. "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead". In: *Nature Machine Intelligence* 1.5 (2019), pp. 206–215. DOI: http://dx.doi.org/10.1038/s42256-019-0048-x.

[17] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. "Methods for Visual Understanding of Hierarchical System Structures". In: *IEEE Transactions on Systems, Man, and Cybernetics* 11.2 (1981), pp. 109–125. DOI: http://dx.doi.org/10.1109/TSMC.1981.4308636.

[18] U.S. Energy Information Administration. *How much electricity does an American home use?* U.S. Energy Information Administration, Frequently Asked Questions. 2024. URL: https://www.eia.gov/tools/faqs/faq.php?id=97&t=3.

[19] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[20] Alex De Vries. "The growing energy footprint of artificial intelligence". In: *Joule* 7.10 (2023), pp. 2191–2194. DOI: http://dx.doi.org/10.1016/j.joule.2023.09.004.

[21] Jie Zhou et al. "Graph Neural Networks: A Review of Methods and Applications". In: *AI Open* 1 (2020), pp. 57–81. DOI: http://dx.doi.org/10.1016/j.aiopen.2021.01.001.